Student Handout — Git & GitHub for DevOps

All Exercises with Solutions

Works with all three approaches (Linear, Project-Driven, Story-Mode)

How to Use This Handout

1. **During class:** Try each exercise on your own first.
2. **When stuck:** Peek at the solution. Don't copy-paste without understanding.
3. **After class:** Re-do the exercises without looking. That's when learning happens.

Every exercise includes:
- The scenario
- The commands to run
- The expected output
- The explanation of WHY it works

SESSION 1 EXERCISES: Your First Repository

Exercise 1.1: Create and Commit a New File

Scenario: You are starting a new project. Create a file called `tools.txt` that lists your DevOps tools.

Environment: Terminal (Git Bash / Terminal)

Task:
1. Navigate to your project folder.
2. Create `tools.txt` with the content: `Git, GitHub, CI/CD`
3. Stage it.
4. Commit it with the message: `Add tools list`
5. Check your history.

Solution:

```
$ echo "Git, GitHub, CI/CD" > tools.txt
$ git add tools.txt
$ git commit -m "Add tools list"
```

Expected Output:

```
[main 1234abc] Add tools list
 1 file changed, 1 insertion(+)
 create mode 100644 tools.txt
```

Verify:

```
$ git log --oneline
```

Expected Output:

```
1234abc Add tools list
def5678 Add course description
abc1234 Add welcome message
```

Why this works:
- `echo > file` writes text to a file.
- `git add` puts the file in the staging area (the box before the photo).
- `git commit -m` takes the photo and labels it.
- `git log` shows all photos in the album.

Exercise 1.2: Fix a File and Commit Again

Scenario: You realize `tools.txt` should also include `Docker`.

Task:
1. Append `Docker` to `tools.txt`.
2. Stage and commit with message: `Add Docker to tools list`
3. Check `git log --oneline`.

Solution:

```
$ echo "Docker" >> tools.txt
$ cat tools.txt
```

Expected Output:

```
Git, GitHub, CI/CD
Docker
```

```
$ git add tools.txt
$ git commit -m "Add Docker to tools list"
```

Verify:

```
$ git log --oneline
```

Why this works:
- `>>` appends (adds to the end). `>` would overwrite.
- Every commit is a snapshot. Git remembers the old version AND the new version.

Exercise 1.3: Check What Changed

Scenario: You want to see what your last commit did.

Task:
1. Show the last commit's details.
2. Show what files changed in the last commit.

Solution:

```
$ git show HEAD
```

To see only files:

```
$ git show HEAD --stat
```

Why this works:
- `HEAD` means "the most recent commit."
- `git show` displays the commit message, author, date, and the diff.
- `--stat` shows only filenames and line counts.

SESSION 2 EXERCISES: Branching and Merging

Exercise 2.1: Create a Feature Branch

Scenario: You want to add a `contact.txt` page. Do it on a branch.

Task:
1. Create and switch to branch `feature-contact`.
2. Create `contact.txt` with: `Email: team@devops.com`
3. Stage and commit.
4. Switch back to `main`.
5. Verify `contact.txt` does NOT exist on `main`.

Solution:

```
$ git branch feature-contact
$ git switch feature-contact
$ echo "Email: team@devops.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"
```

Verify main is safe:

```
$ git switch main
$ ls
```

Expected Output: (NO `contact.txt`)

```
about.txt  home.txt  tools.txt
```

Why this works:
- Branches are parallel universes. Changes on `feature-contact` do not affect `main`.
- This is the #1 reason branches exist — to protect `main`.

Exercise 2.2: Merge the Branch

Scenario: The contact page is ready. Merge it into `main`.

Task:
1. Merge `feature-contact` into `main`.
2. Verify `contact.txt` now exists.
3. Delete the branch.

Solution:

```
$ git switch main
$ git merge feature-contact
```

Expected Output:

```
Fast-forward
 contact.txt | 1 +
 1 file changed, 1 insertion(+)
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  home.txt  tools.txt
```

Delete branch:

```
$ git branch -d feature-contact
```

Why this works:
- `git merge` moves changes from one branch to another.
- Fast-forward means `main` was unchanged since branching — clean and simple.
- Deleting branches keeps things tidy. The commits are safely in `main`.

Exercise 2.3: Create and Merge Multiple Branches

Task:
1. Create branch `feature-skills` and add `skills.txt`.
2. Merge it.
3. Create branch `feature-services` and add `services.txt`.
4. Merge it.
5. Check `git log --oneline`.

Solution:

```
$ git branch feature-skills
$ git switch feature-skills
$ echo "Git, Docker, Kubernetes" > skills.txt
$ git add skills.txt
$ git commit -m "Add skills page"
$ git switch main
$ git merge feature-skills
$ git branch -d feature-skills

$ git branch feature-services
$ git switch feature-services
$ echo "Web Hosting, CI/CD" > services.txt
$ git add services.txt
$ git commit -m "Add services page"
$ git switch main
$ git merge feature-services
$ git branch -d feature-services
```

Verify:

```
$ git log --oneline
$ ls
```

Why this works:
- Each branch is independent. You merge them when ready.
- This is how teams work: multiple developers, multiple branches, one `main`.

SESSION 3 EXERCISES: GitHub and Pull Requests

Exercise 3.1: Push to GitHub

Scenario: Your code is ready. Push it to GitHub.

Task:
1. Add remote `origin` pointing to your GitHub repo.
2. Push `main`.
3. Refresh GitHub and verify files are there.

Solution:

```
# Replace YOURUSERNAME with your actual username
$ git remote add origin git@github.com:YOURUSERNAME/my-repo.git
$ git push -u origin main
```

Expected Output:

```
 * [new branch]      main -> main
```

Verify:
- Open `https://github.com/YOURUSERNAME/my-repo` in your browser.
- You should see your files.

Why this works:
- `origin` is a nickname for your GitHub repo. You only set it once.
- `-u` links your local `main` to remote `main`. After this, just `git push` works.

Exercise 3.2: Complete PR Workflow

Scenario: Add a `faq.txt` file using the full Pull Request workflow.

Task:
1. Create branch `feature-faq`.
2. Add `faq.txt` with a question and answer.
3. Commit and push.
4. Open a Pull Request on GitHub.
5. Review, approve, and merge it.
6. Pull locally.
7. Delete the branch.

Solution:

```
$ git branch feature-faq
$ git switch feature-faq
$ cat > faq.txt << 'EOF'
Q: What is this project?
A: My DevOps portfolio.
EOF
$ git add faq.txt
$ git commit -m "Add FAQ page"
$ git push -u origin feature-faq
```

On GitHub:
1. Click **"Compare & pull request"**
2. Title: `Add FAQ page`
3. Create PR
4. **Files changed** → Review
5. **Review changes** → Approve → Submit
6. **Merge pull request** → Confirm
7. **Delete branch**

Back in terminal:

```
$ git switch main
$ git pull
$ git branch -d feature-faq
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  faq.txt  home.txt  services.txt  skills.txt
```

Why this works:
- PRs are the professional way to merge code. No one pushes directly to `main`.
- Code review catches bugs before they reach production.

Exercise 3.3: Pull Updates from Teammates

Scenario: A teammate pushed a change to `main` on GitHub.

Task:
1. Run `git pull` to get their changes.
2. Check `git log --oneline` to see their commit.

Solution:

```
$ git switch main
$ git pull
```

Why this works:
- `git pull` downloads changes from GitHub AND merges them into your local `main`.
- Always pull before starting new work. Otherwise you'll work on old code.

SESSION 4 EXERCISES: Undoing and Conflicts

Exercise 4.1: Revert a Bad Commit

Scenario: You accidentally overwrote `home.txt` with bad content.

Task:
1. Overwrite `home.txt` badly and commit it.
2. Find the bad commit ID.
3. Revert it.
4. Verify the original content is restored.

Solution:

```
$ echo "BAD CONTENT" > home.txt
$ git add home.txt
$ git commit -m "Accidentally broke homepage"
$ git log --oneline
```

Expected: Top line is the bad commit (e.g., `bad1234`).

```
$ git revert bad1234
```

An editor opens. Save and exit.

Expected Output:

```
[main revert789] Revert "Accidentally broke homepage"
```

Verify:

```
$ cat home.txt
```

Expected: Original content is back.

Why this works:
- `git revert` creates a NEW commit that undoes the bad one.
- The bad commit stays in history (audit trail). This is the safe way.

Exercise 4.2: Resolve a Merge Conflict

Scenario: Two branches both edited `about.txt`. Git can't decide which to keep.

Task:
1. Change `about.txt` on `main`.
2. Create branch `update-about-1` from BEFORE that commit and change it differently.
3. Merge both into `main`.
4. Resolve the conflict.
5. Complete the merge.

Solution:

```
# Change about.txt on main
$ echo "Updated main version" > about.txt
$ git add about.txt
$ git commit -m "Update about on main"

# Create branch from before this commit
$ git branch update-about-1 HEAD~1
$ git switch update-about-1
$ echo "Updated branch version" > about.txt
$ git add about.txt
$ git commit -m "Update about on branch"

# Merge into main
$ git switch main
$ git merge update-about-1
```

Expected: `CONFLICT (content): Merge conflict in about.txt`

Resolve:

```
$ cat about.txt
# (see conflict markers)
```

Edit `about.txt` — remove `<<<<<<<`, `=======`, `>>>>>>>` lines, keep the content you want:

```
Main version updated with branch improvements.
```

```
$ git add about.txt
$ git commit -m "Resolve about page conflict"
```

Why this works:
- Conflicts mean Git needs human judgment. It's not broken — it's asking for help.
- After editing, you MUST stage and commit to complete the merge.

Exercise 4.3: Use Git Stash

Scenario: You're editing a file but need to switch tasks urgently.

Task:
1. Start editing a file (but don't commit).
2. Stash your work.
3. Do something else on `main`.
4. Pop your stash back.

Solution:

```
$ git switch main
$ echo "Work in progress..." >> about.txt
$ git stash
```

Expected Output:

```
Saved working directory and index state...
```

Do something else:

```
$ git status
# (clean)
```

Restore work:

```
$ git stash pop
```

Expected Output:

```
On branch main
Changes not staged for commit:
  modified:   about.txt
```

Why this works:
- Stash saves your uncommitted work temporarily.
- The working directory becomes clean so you can switch branches or fix emergencies.

SESSION 5 EXERCISES: CI/CD with GitHub Actions

Exercise 5.1: Create a CI Workflow

Scenario: You want a robot to check that `home.txt` and `about.txt` exist on every push.

Task:
1. Create `.github/workflows/check-files.yml`.
2. Write a workflow that checks files exist.
3. Commit and push.
4. Watch it pass in the Actions tab.

Solution:

```
$ mkdir -p .github/workflows
$ cat > .github/workflows/check-files.yml << 'EOF'
name: Check Files
on:
  push:
    branches: [ main ]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Check required files
        run: |
          test -f home.txt && echo "□ home.txt"
          test -f about.txt && echo "□ about.txt"
          echo "All checks passed!"
EOF
$ git add .github/workflows/check-files.yml
$ git commit -m "Add CI workflow"
$ git push
```

Verify:
- Go to GitHub → **Actions** tab
- See green checkmark □

Why this works:
- YAML files in `.github/workflows/` are automatically detected by GitHub.
- The `actions/checkout@v4` step downloads your code onto the runner.
- `run:` executes shell commands.

Exercise 5.2: Watch CI Catch a Bug

Scenario: You intentionally break something and watch CI fail.

Task:
1. Create branch `break-ci`.
2. Delete `about.txt`.
3. Commit and push.
4. Open a PR and watch CI fail □.
5. Close the PR without merging.

Solution:

```
$ git branch break-ci
$ git switch break-ci
$ git rm about.txt
$ git commit -m "Remove about page (should fail CI)"
$ git push -u origin break-ci
```

On GitHub:
1. Open PR
2. CI runs automatically
3. Fails with "□ about.txt MISSING"
4. Close PR

Why this works:
- CI catches bugs BEFORE they reach `main`.
- Failed PRs should never be merged.

Exercise 5.3: Set Up Branch Protection

Scenario: Lock `main` so no one can push directly.

Task:
1. Add branch protection rule for `main`.
2. Require PR before merging.
3. Require status checks to pass.
4. Try direct push and verify you're blocked.

Solution:

On GitHub:
1. Repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
- Require pull request before merging
- Require status checks to pass
- Select `Check Files`
4. **Create**

Test in terminal:

```
$ git switch main
$ echo "test" >> home.txt
$ git add home.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — protected branch.

Undo test commit:

```
$ git reset --soft HEAD~1
$ git restore --staged home.txt
$ git checkout -- home.txt
```

Why this works:
- Branch protection enforces quality gates.
- No one can accidentally break production by pushing directly.

BONUS EXERCISES

Bonus 1: Write a Good `.gitignore`

Scenario: You never want to commit secrets, logs, or IDE files.

Task:
1. Create `.gitignore`.
2. Add entries for `.env`, `*.log`, `.vscode/`.
3. Commit it.

Solution:

```
$ cat > .gitignore << 'EOF'
# Secrets
.env
.env.local

# Logs
*.log
logs/

# IDE
.vscode/
.idea/
EOF
$ git add .gitignore
$ git commit -m "Add .gitignore"
```

Why this works:
- `.gitignore` tells Git to never track matching files.
- **Never commit secrets.** Use `.gitignore` to protect yourself.

Bonus 2: View Blame

Scenario: A line of code is confusing. Who wrote it?

Task:
1. Run `git blame` on a file.
2. Identify who wrote each line.

Solution:

```
$ git blame home.txt
```

Expected Output:

```
abc1234 (Your Name  2024-01-15) Welcome to my portfolio!
def5678 (Your Name  2024-01-16) I'm learning DevOps.
```

Why this works:
- `git blame` shows the commit and author for every line.
- Essential during incidents: "Who changed this config line?"

Bonus 3: Tag a Release

Scenario: Your portfolio is complete. Tag it as version 1.0.0.

Task:
1. Create annotated tag `v1.0.0`.
2. Push it to GitHub.
3. View it on GitHub.

Solution:

```
$ git tag -a v1.0.0 -m "First stable release"
$ git push origin v1.0.0
```

Verify:
- GitHub → **Releases** (right sidebar)
- See your tag

Why this works:
- Tags mark specific commits as milestones.
- Semantic versioning (v1.0.0) is the industry standard.

End of Student Handout — Good luck, engineer!